

BetON — Refactoring AI

Sprzątanie martwego kodu w serwisie symulacji meczu

Folder	ai/
Stack	FastAPI + Pydantic + structlog
Zakres	Usunięcie LLM/RAG/dead endpoints; refactor schemas, tools, main
Wynik	711 linii Pythona (z ~2000), 6 paczek w requirements (z 35)
Endpointy live	POST /generate/match, POST /odds/live
Data	2026-06-07

1. Po co to wszystko

Folder `ai/` startował jako prototyp z trzema warstwami: silnik symulacji meczu, system RAG z indeksem FAISS i historią Premier League z 11 plików Excel, oraz integracja LLM (OpenAI / Gemini / lokalny stub) z function-callingiem. Wraz z rozwojem projektu BetON, produkcyjny backend Node.js zaczął używać **wyłącznie dwóch matematycznych endpointów**: jednego do generacji symulacji meczu, drugiego do przeliczania kursów na żywo. Pozostałe warstwy (LLM, RAG, function-calling) nie były wołane przez nikogo — zostały tylko jako balast.

Cel refactoringu to **clean code**: zostawić tylko to co działa, wywalić resztę. Konsekwencje techniczne: startup AI w ~1s zamiast ~15s (bez ładowania modelu embeddingów i indeksu FAISS), instalacja zależności w sekundach zamiast minut, image Dockera mniejszy o ~750 MB, czytelne moduły bez funkcji-zombie.

2. Jak to teraz działa

2.1. Trzy warstwy projektu BetON

BetON składa się z trzech procesów wpiętych w jedno doświadczenie użytkownika:

Warstwa	Język / framework	Rola
frontend/	React + Vite + Tailwind	UI dla użytkownika — lista meczów, szczegóły meczu z relacją live, kupon (bet builder), historia kuponów z cashoutem.
backend/	Node.js + Express + Prisma (MySQL)	Persystencja meczów/kuponów, autoryzacja JWT, scheduler importu i live odds, settlement i cashout.
ai/	Python + FastAPI	Czysty silnik matematyczny: symulacja przebiegu meczu, kalkulator kursów przedmeczowych i live.

2.2. Tor przepływu danych

Życie meczu od wygenerowania do rozliczenia kuponu:

- **Import** — backend co tick (lub na starcie) generuje pary drużyn, woła `POST ai:/generate/match` dla pierwszego meczu synchronicznie (`page=1`) i zapisuje go do MySQL razem z 24 kursami; kolejne mecze lecą w tle.
- **Pre-match** — frontend pobiera `GET /api/matches`, pokazuje kursy 1X2 + dodatkowe rynki (OU bramek, BTTS, OU różnych, OU kartek, OU celnych strzałów, OU spalonych, HT/FT).
- **Live** — gdy mecz osiąga datę startu, backend cron co 5s wywołuje `POST ai:/odds/live` z aktualnym snapshotem statystyk (wynik, minuta, kartki, różne, strzały, spalone). AI zwraca 15 nowych kursów. Backend zapisuje je z `next_update_at = now + 30s`; 5s przed kolejnym tickiem ustawia status ZABLOKOWANY (użytkownik nie może postawić).
- **Settlement** — po minucie 90 cron porównuje wynik końcowy z każdym kursem (1X2, OU_GOALS, BTTS, OU_CORNERS, OU_CARDS, OU_SOT, OU_OFFSIDES, HTFT) i ustawia WIN/LOSS/VOID. Kupony z samymi WIN wypłacane na saldo.
- **Cashout** — w trakcie meczu użytkownik może wypłacić kupon wcześniej. Backend liczy: $\text{stake} \times \prod(\text{multiplier}) \times 0.90$, gdzie $\text{multiplier per pozycja} = \text{kurs}_w_{\text{momencie}} / \text{kurs}_{\text{aktualny}}$ dla pending.

2.3. Granica ai/ ↔ backend/

Po refactorze AI komunikuje się z resztą świata **wyłącznie** przez 4 routy: dwa biznesowe i dwa diagnostyczne. Zero zewnętrznych zależności (LLM, RAG), wszystkie obliczenia są deterministyczne z lekkim losowym jitterem.

3. Co zostało skasowane

Audyt całego folderu ai/ pod kątem reachability z jedynych dwóch żywych endpointów wykazał, że poniższe moduły, pliki i foldery były **nieosiągalne** z punktu widzenia produkcji. Wszystkie skasowane.

Ścieżka	Powód usunięcia
ai/app/llm_service.py	OpenAI/Gemini/Local providers — używane tylko przez /ask (nikt nie woła).
ai/app/rag_service.py	Indeksowanie FAISS + sentence-transformers — używane tylko przez /ask + lifespan.
ai/app/guardrails.py	Prompt-injection detection, rate limiting — tylko /ask. Funkcja validate_team_name inlined do tools.py.
ai/generate_demo_report.py ai/DEMO_REPORT.md	Skrypt demo strzelał wyłącznie w /ask. Raport zdezaktualizowany.
ai/DANE/ (~13 MB)	11 plików Excel z historią Premier League — czytane tylko przez RAG.
ai/data/ (FAISS + JSON)	Cache indeksu wektorowego dla RAG.
ai/tests/	Testy /ask, RAG.retrieve, ToolDispatcher, prompt-injection — wszystko martwe ścieżki.
ai/output/ (~1 MB)	Stare dumpy JSON z ręcznych testów /generate/batch.

3.1. Endpointy wycięte z main.py

Endpoint	Co robił	Status
POST /ask	LLM Q&A; z RAG i function-callingiem.	USUNIĘTY
POST /generate/batch	Generacja wielu meczów naraz.	USUNIĘTY (zastąpiony pagination)
GET /teams	Lista drużyn z bazy historycznej.	USUNIĘTY
GET /teams/{name}/stats	Statystyki drużyny z RAG.	USUNIĘTY
GET /tools	Lista zarejestrowanych narzędzi LLM.	USUNIĘTY
GET /metrics	Liczniki requestów / tool calls.	USUNIĘTY

3.2. requirements.txt: z 35 paczek do 6

Wycięte zależności (sumarycznie ~750 MB instalacji):

- **LLM:** openai, google-generativeai
- **RAG/embeddings:** sentence-transformers, faiss-cpu, numpy
- **Data:** pandas, openpyxl
- **HTTP klient:** httpx, requests
- **Auth/multipart:** python-jose[cryptography], python-multipart, tenacity
- **Testy:** pytest, pytest-asyncio, pytest-cov

4. Co przeżyło i jak jest zbudowane

Plik	Linie	Rola
app/__init__.py	4	Marker pakietu + wersja.
app/config.py	21	Settings: api_host, api_port, log_level.
app/main.py	207	FastAPI app + 4 endpointy + exception handlery.
app/schemas.py	202	Pydantic modele: MatchEventType, MatchEvent, MatchMinuteData, PreMatchOdds, MatchSimulation, GenerateMatchRequest, LiveOddsRequest, APIResponse.
app/tools.py	271	generate_match_simulation + _generate_minute_by_minute + _compute_htft_odds + validate_team_name.
requirements.txt	6	fastapi, uvicorn[standard], pydantic, pydantic-settings, python-dotenv, structlog.
Dockerfile	20	python:3.11-slim + curl + uvicorn. HEALTHCHECK na /health.
docker-compose.yml	12	Pojedynczy serwis 'api' na porcie 8000.
README.md	—	Opis 2 endpointów + curl examples + struktura.

5. Kontrakt API

5.1. POST /generate/match

Wejście (GenerateMatchRequest):

```
{
  "home_team": "Liverpool", // 1-50 zn., [\w\s\-\.\.]
  "away_team": "Arsenal",
  "date": "2026-06-07", // opcjonalne YYYY-MM-DD
  "use_historical_data": true // pole zachowane dla kompatybilności
}
```

Wyjście (MatchSimulation):

- **match_id, home_team, away_team, date, kick_off_time, stadium, referee** — metadane
- **pre_match_odds** — 32 pola: 1X2, OU bramek (linia 2.5), BTTS, OU różnych (linia 9.5), OU kartek (linia 4.5), OU celnych strzałów (linia 8.5), OU spalonych (linia 3.5), 9× HT/FT, Asian Handicap
- **minutes** — tablica 91 obiektów MatchMinuteData (minuty 0–90), każdy z: wynikiem, posiadaniem, komentarzem, listą eventów + 14 statystyk kumulatywnych (strzały, różne, faule, kartki, spalone)
- **final_score_home, final_score_away** — wynik końcowy

5.2. POST /odds/live

Wejście (LiveOddsRequest) — aktualny snapshot stanu meczu:

```
{
  "home_team": "Liverpool", "away_team": "Arsenal",
  "minute": 60,
  "home_score": 1, "away_score": 0,
  "home_corners": 4, "away_corners": 2,
  "home_yellow_cards": 1, "away_yellow_cards": 2,
  "home_red_cards": 0, "away_red_cards": 0,
  "home_shots_on_target": 5, "away_shots_on_target": 2,
  "home_offsides": 1, "away_offsides": 2,
  "goals_line": 2.5, "corners_line": 9.5,
  "cards_line": 4.5, "sot_line": 8.5, "offsides_line": 3.5
}
```

Wyjście — 15 kursów:

```
{
  "home_win": 1.66, "draw": 4.06, "away_win": 5.26,
  "over_goals": 3.11, "under_goals": 1.37,
  "btts_yes": 3.06, "btts_no": 1.38,
  "over_corners": ..., "under_corners": ...,
  "over_cards": ..., "under_cards": ...,
  "over_sot": ..., "under_sot": ...,
  "over_offsides": ..., "under_offsides": ...
}
```

6. Jak działa silnik

6.1. generate_match_simulation()

Funkcja zwracająca pełną symulację meczu jako słownik. Pipeline:

- **Walidacja drużyn** — regex `^[\\w\\s\\-\\.]{1,50}$` (blokada path-traversal i nadmiernych znaków specjalnych).
- **Bazowe prawdopodobieństwa 1X2** — losowa siła gospodarza w $[0.4, 0.6]$, $\text{home_win_prob} = \text{strength} \times 0.5 + 0.2$, $\text{draw_prob} = 0.25$, $\text{away_win_prob} = \text{reszta}$.
- **Kursy przedmeczowe** — z prawdopodobieństw ($\text{kurs} = 0.95 / p$, margines 5%); OU/BTTS/SOT/spalone z losowych zakresów 1.7–2.1 (rynkı zbalansowane). HTFT z helpera `_compute_htft_odds`.
- **Symulacja minuta po minucie** — pętla 0–90: w każdej minucie losowane zdarzenie z dystrybucją: gol (3–4%), strzał (12%), rożny (7%), faul (8%), spalony (6%), pozostałe — komentarz neutralny. Żółta kartka wpada z prawdopodobieństwem 20% każdego faulu. Statystyki narastają, posiadanie dryfuje w korytarzu 35–65%.
- **Walidacja Pydantic** — wynik konwertowany do MatchSimulation, co waliduje wszystkie zakresy pól (np. `final_score` 0–20, `kurs` > 1.0).

6.2. _compute_htft_odds()

Heurystyka kursów dla rynku 1. połowa / mecz (9 kombinacji). Założenie: HT ma większy udział remisu (krótszy czas = mniej rozstrzygnięć), więc prawdopodobieństwa HT_home/HT_away są skalowane przez 0.55. Wyniki 'spójne' (1/1, X/X, 2/2) dostają bonus probabilistyczny $\times 1.6$ — bo realnie drużyna prowadząca do przerwy częściej wygrywa mecz. Margines bukmacherski 8% ($\text{kurs} = 0.92 / p$).

6.3. /odds/live — model logistyczny

Każdy rynek liczony osobno, ale wspólnym mechanizmem:

- **1X2** — bonus za prowadzenie skaluje się z postępem meczu: $\text{home_p} = 0.45 + 0.20 \times \text{diff} \times (0.4 + 0.6 \times \text{progress})$. Remis dostaje boost przy $\text{diff}=0$ w końcówce meczu (drużyny przestają ryzykować).
- **BTTS** — jeśli oba zespoły strzeliły, $\text{YES} = 0.98$. Inaczej $P(\text{strzelą do końca}) = 1 - \exp(-1.3 \times \text{remaining})$ per zespół, niezależnie.
- **OU rynki (goals/corners/cards/SOT/offsidess)** — wspólny wzór: $\text{expected} = \text{aktualne} + \text{tempo} \times \text{remaining}$ ($\text{tempo} \sim \text{średnia z meczu}$), potem $z = (\text{expected} - \text{line}) / \text{scale}$, $P(\text{over}) = 1 / (1 + \exp(-z))$. Funkcja logistyczna trzyma kursy w sensownym zakresie.
- **Konwersja P → kurs:** $\text{base} = 0.95 / p$, potem dodaje jitter $\text{uniform}(-0.04, +0.04)$, żeby kolejne ticki nie powtarzały tego samego kursu (efekt 'żywych' kursów).

7. Liczby — przed i po

Metryka	Przed	Po	Zmiana
Linie Python (app/)	~2 000	705	-65%
Plików .py	8	5	-3
Endpointy FastAPI	10	4	-6
Paczki w requirements.txt	35	6	-29
Rozmiar pip install (~)	~750 MB	~10 MB	-740 MB
DANE/ (Excel)	~13 MB	0	całość
Foldery skasowane	tests/, data/, DANE/, output/	—	4 foldery
Czas startu uvicorn	~15 s (FAISS+embeddingi)	< 1 s	~15× szybciej
Klasy LLM/RAG	4 providery, RAGService, ToolRegistry, ToolDispatcher	0	100%

8. Weryfikacja

Po cleanup'ie sprawdzono:

- `python -c "from app.main import app"` — brak ImportError
- Lista routów: `['/', '/health', '/generate/match', '/odds/live']` + auto-routy FastAPI (openapi, docs, redoc)
- `generate_match_simulation('Liverpool', 'Arsenal')` → 91 minut, 32 pola kursów, walidacja Pydantic przechodzi
- `GET /` → 200, { status: ok, version: 2.0.0 }
- `GET /health` → 200, { status: healthy, timestamp }
- `POST /generate/match` → 200, kompletna struktura z 91 minutami
- `POST /odds/live` → 200, dokładnie 15 kursów
- `Path-traversal w home_team` → 422 (walidacja Pydantic chwyta `'../../etc/passwd'`)

8.1. Weryfikacja integracji z backend

Po refactorze sprawdzono grepem wszystkie odwołania do `localhost:8000` w całym repo. Backend uderza w AI wyłącznie z:

- `backend/services/matchImporter.js:6` → `/generate/match` (page-by-page import)
- `backend/services/liveOddsService.js:4` → `/odds/live` (cron co 5s)

Frontend (`frontend/src/api/client.js`) komunikuje się wyłącznie z backend Node (port 5001) — zero bezpośrednich wywołań do AI. Payloady wysyłane przez backend zgadzają się 1:1 z polami w zaktualizowanym

GenerateMatchRequest i LiveOddsRequest; pola odpowiedzi konsumowane przez backend (15 kursów live + ~30 pól pre-match + 14 statystyk per minuta + 91 minut) pokrywają się dokładnie z tym co zwraca nowy main.py.

Wniosek: cleanup nie wpłynął na żaden żywy code path. Backend i frontend działają identycznie jak przed refactorom, ale serwis AI startuje szybciej, jest mniejszy i prostszy w utrzymaniu.

9. Co warto rozważyć w przyszłości

- **Testy** — folder tests/ został skasowany. Warto dorobić ~3 testy pytest dla /generate/match (struktura odpowiedzi), /odds/live (deterministyczność przy seedowanym randomie) i walidacji wejścia.
- **Seed dla randomu** — obecnie symulacja jest losowa. Można dodać parametr seed do GenerateMatchRequest dla powtarzalnych testów i debugowania.
- **Caching live odds** — przy dużej liczbie meczów backend może spamować /odds/live co 5s × N meczów. Można dorobić w-memory cache w AI z TTL 4s, żeby nie liczyć tego samego dwa razy w tym samym tick'u.
- **Async w AI** — obecnie endpointy są deklarowane async ale funkcje wewnętrzne są synchroniczne. Dla obecnej skali jest to OK (symulacja zajmuje ~50 ms), ale w razie wzrostu można zrobić proper async lub przerzucić ciężki random na thread pool.
- **Real Poisson** — obecny model goli to ruletka z prawdopodobieństwem 3-4% per minuta. Klasyczne podejście to model Poissona z parametrem λ zależnym od siły drużyn (np. Dixon-Coles). Można podmienić w _generate_minute_by_minute bez zmian w API.

Raport wygenerowany 2026-06-07 ze stanu repozytorium po finalnym refactoringu folderu ai/.